Diseño y desarrollo de servicios web - caso

Servicios de registro e inicio de sesión

GA7-220501096-AA5-EV01

Laura Tatiana Ciro Botero

Servicio Nacional de Aprendizaje - SENA

Tecnología en Análisis y Desarrollo de Software

Ficha 3235898

Instructora Adriana Milena Rangel

2026

Introducción y objetivos

Un servicio web permite que diferentes aplicaciones utilicen una misma función mediante una petición. En este caso se desarrollan dos métodos reutilizables: registrar una cuenta y verificar el usuario y la contraseña para el inicio de sesión. La pantalla de práctica consume esos métodos; las reglas se ejecutan en el servidor.

La guía 7 solicita diseñar y codificar el caso de registro e inicio de sesión, devolver un mensaje de autenticación satisfactoria o error, incluir comentarios y utilizar versionamiento. La entrega debe contener los archivos del proyecto y un archivo con el enlace del repositorio dentro de una carpeta comprimida (Servicio Nacional de Aprendizaje [SENA], s. f., p. 14).

Objetivo general

Implementar servicios web reutilizables de registro y autenticación con Java y Spring Boot, verificando los datos recibidos y documentando su funcionamiento y versionamiento.

Objetivos específicos

• Definir las peticiones y respuestas de los servicios.

• Registrar usuarios únicos y proteger sus contraseñas.

• Comprobar credenciales y comunicar el resultado correcto.

• Verificar errores y conservar el código comentado en Git y GitHub.

Alcance: caso académico independiente del sistema completo de selección. La autenticación confirma las credenciales y devuelve el mensaje exigido; no emite tokens ni crea una sesión con acceso a Recursos Humanos. La prueba con video en Postman corresponde a AA5-EV02, no a esta entrega.

Diseño del servicio y organización

El controlador recibe la petición HTTP y el JSON. El servicio aplica las reglas y utiliza una consulta parametrizada para comunicarse con la base de datos. Spring convierte las respuestas Java a JSON para que otro programa pueda utilizarlas (Spring, s. f.-a).

Figura 1

Arquitectura implementada



Nota. Elaboración propia. Ambos servicios comparten el contrato Credenciales, las reglas de validación y la misma base de cuentas.

Archivo o elemento	Responsabilidad
Credenciales.java	Define usuario y contrasena; valida obligatoriedad y longitud.
AuthController.java	Publica POST /api/registro y POST /api/login.
AuthService.java	Normaliza el usuario, registra y compara contraseñas.
ErroresApi.java	Convierte validaciones y conflictos en respuestas comprensibles.
schema.sql	Crea la tabla usuario con restricción UNIQUE.
static/index.html y app.js	Permiten practicar y ver el resultado del servicio.

La base guarda id, nombre_usuario, password_hash y fecha_registro. El nombre se conserva normalizado y es único. H2 persiste en la carpeta data al ejecutar normalmente; las pruebas usan otra base en memoria.

Servicio de registro

Método: **POST**. Ruta: **/api/registro**. El cuerpo usa Content-Type: application/json y recibe exactamente los campos usuario y contrasena. El siguiente ejemplo es ficticio:

```
{"usuario":"tatiana.demo","contrasena":"Practica2026!"}
```

Paso a paso

1. Recibir el JSON y comprobar los campos obligatorios y las longitudes.
2. Quitar espacios exteriores del usuario y convertirlo a minúsculas.
3. Validar su formato y el límite de bytes de la contraseña.
4. Crear el hash BCrypt y guardar el usuario mediante SQL parametrizado.
5. Devolver HTTP 201; si el nombre ya existe, devolver HTTP 409.

Fragmento del servicio implementado

```
String usuario = validar(datos);
String hash = encoder.encode(datos.contrasena());
// UNIQUE impide registrar el mismo nombre dos veces.
db.update(
    "INSERT INTO usuario(nombre_usuario, password_hash)"
    + " VALUES (?, ?)", usuario, hash
);
```

Respuesta de registro correcto: HTTP 201

```
{
  "estado": "registro_exitoso",
  "mensaje": "Usuario registrado correctamente.",
  "usuario": "tatiana.demo"
}
```

La restricción UNIQUE mantiene la regla incluso si dos peticiones intentan registrar el mismo usuario simultáneamente. La respuesta no contiene la contraseña ni su hash.

Servicio de inicio de sesión

Método: **POST**. Ruta: **/api/login**. Recibe los mismos dos campos. Primero verifica el formato; después consulta el hash asociado al usuario y compara la contraseña recibida mediante PasswordEncoder.matches.

BCrypt transforma la contraseña en un hash con sal aleatoria. No se descifra ese hash; se comprueba si la contraseña proporcionada coincide con él. Esta es la función del codificador de contraseñas de Spring (Spring, s. f.-b).

Decisión principal del servicio

```
boolean coincide = encoder.matches(datos.contrasena(), hash);
if (hashes.isEmpty() || !coincide) {
    throw new ResponseStatusException(
        HttpStatus.UNAUTHORIZED,
        "Error en la autenticación."
    );
}
```

Autenticación correcta: HTTP 200

```
{
  "estado": "autenticacion_satisfactoria",
  "mensaje": "Autenticación satisfactoria.",
  "usuario": "tatiana.demo"
}
```

Autenticación incorrecta: HTTP 401

```
{
  "estado": "error",
  "mensaje": "Error en la autenticación."
}
```

Se usa el mismo mensaje para usuario inexistente y contraseña incorrecta. También se realiza una comparación BCrypt cuando el usuario no existe. El resultado del caso no concede permisos adicionales.

Validaciones y respuestas de verificación

Tabla 1

Reglas aplicadas en el servidor

Condición	Comportamiento
Usuario o contraseña vacíos	HTTP 400 con indicaciones de corrección.
Nombre del usuario	Entre 3 y 30 letras ASCII, números, punto, guion o guion bajo.
Mayúsculas y espacios del usuario	Se convierte a minúsculas y se quitan espacios exteriores antes de registrar o buscar.
Contraseña	Entre 8 y 64 caracteres, sin quitar sus espacios ni modificar mayúsculas.
Límite BCrypt	Máximo 72 bytes UTF-8; se rechaza el exceso, nunca se trunca.
Usuario repetido	HTTP 409. La base tiene una restricción UNIQUE.
JSON mal formado o campos adicionales	HTTP 400; no se admiten campos de rol o permisos.
Credenciales válidas que no coinciden	HTTP 401 con mensaje genérico de error de autenticación.
Registro válido	HTTP 201 y confirmación del registro.
Credenciales correctas	HTTP 200 y confirmación de autenticación.

Las consultas usan parámetros en lugar de concatenar los valores del usuario. Los errores muestran instrucciones, pero no reproducen contraseñas. El cliente no registra las claves en la consola y las borra del campo después de una operación correcta.

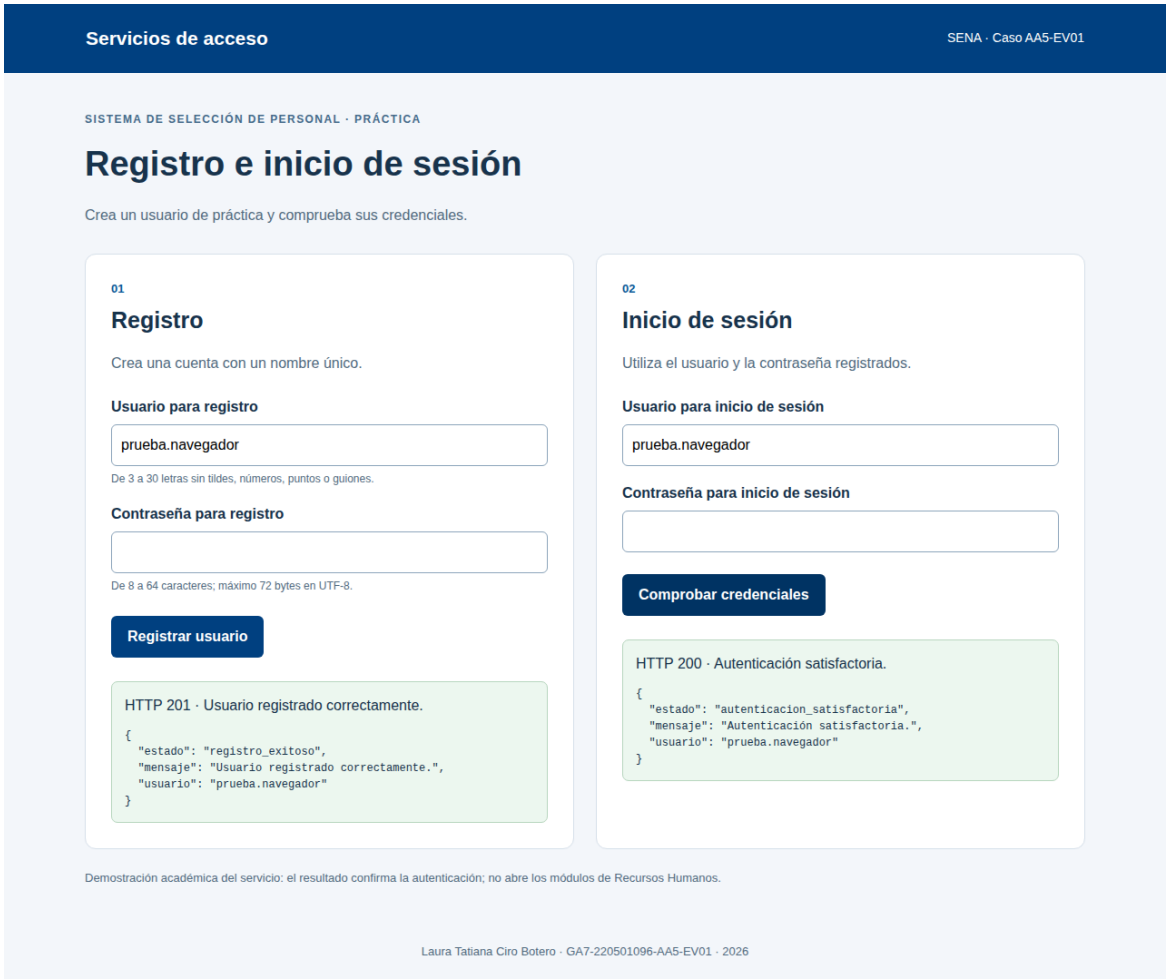
Estas reglas resuelven el caso de la guía. Un despliegue con usuarios reales requeriría, además, HTTPS, control de intentos y una gestión completa de sesiones y recuperación de cuentas.

Ejecución del registro y autenticación correcta

La pantalla se ejecutó contra el servidor Spring Boot real. Primero se registró prueba.navegador y después se enviaron sus credenciales al servicio de inicio de sesión. El resultado visible corresponde a las respuestas HTTP 201 y HTTP 200.

Figura 2

Registro y autenticación satisfactoria



Nota. Captura real de Chromium durante la ejecución en GitHub Actions. La cuenta y las claves usadas son ficticias. Los datos se guardan en una base H2 aislada para las pruebas. Los campos de contraseña se limpian después de recibir una respuesta correcta.

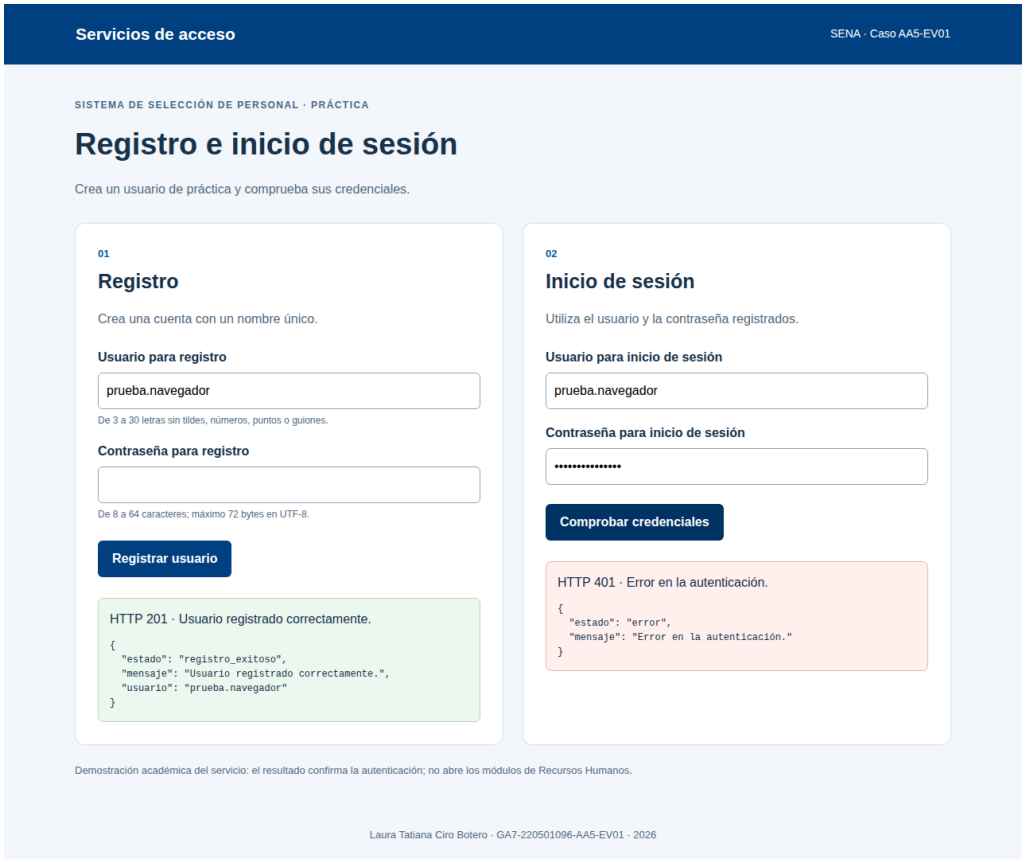
Así se demuestra la reutilización: la pantalla llama al servicio mediante HTTP, y el mismo contrato puede ser utilizado por otro cliente sin copiar las reglas de autenticación.

Ejecución del error de autenticación

Después del inicio de sesión correcto se envió una contraseña distinta. El servidor respondió HTTP 401 con el mensaje de error solicitado. El usuario registrado permaneció en la base; el intento incorrecto no modificó su contraseña.

Figura 3

Respuesta al ingresar una contraseña incorrecta



Nota. Captura real de la prueba automatizada. El campo de contraseña permanece enmascarado. También se verificó el envío de campos vacíos, que devuelve HTTP 400; su captura se incluye como 04-validacion.png dentro de evidencias/capturas.

Un 401 significa que las credenciales no fueron aceptadas. Un 400 significa que la petición debe corregirse antes de intentar la autenticación. Distinguir estos resultados ayuda a que el cliente muestre un mensaje adecuado.

Pruebas y resultados

Se compiló el proyecto con Java 17 y Maven y se ejecutaron las pruebas con H2 en memoria. Después se inició el archivo ejecutable generado y Chromium consumió sus servicios mediante la pantalla de práctica.

Tabla 2

Comprobaciones ejecutadas

Grupo	Casos comprobados	Resultado
Registro	Registro correcto; duplicado normalizado.	2 aprobados
Autenticación	Correcta; contraseña incorrecta; usuario inexistente; usuario con mayúsculas; contraseña con espacios.	5 aprobados
Validación	Usuario vacío; clave corta; caracteres no admitidos; JSON mal formado; campos adicionales; límite UTF-8; login vacío.	7 aprobados
Protección del almacenamiento	El valor guardado es un hash válido; la respuesta excluye contraseña y hash.	1 aprobado
Navegador real	Registro, autenticación correcta, error y validación desde los formularios.	1 prueba aprobada

Resultado total: 16 pruebas aprobadas: 15 de integración y 1 de navegador. Sin fallos ni pruebas omitidas.

Ejecución verificable: Abrir el reporte de GitHub Actions

Los reportes de Maven, el reporte HTML de Playwright y las capturas originales se incluyen en el ZIP. Las pruebas comprueban el alcance del caso; no constituyen una certificación de seguridad de un sistema en producción.

Las bases de pruebas son independientes de la base persistente de práctica. Los registros de prueba no alteran los candidatos ni los datos de otras evidencias.

Versionamiento, repositorio y entrega

Repositorio: sena-ga7-aa5-ev01-servicios

Git registra las versiones mediante commits y GitHub conserva el proyecto y el flujo de comprobación. El código contiene comentarios sobre decisiones relevantes: protección de contraseñas, normalización, consultas parametrizadas y presentación de respuestas sin ejecutar HTML recibido.

Versión comprobada: bcaf666ef306b7e24456ed820d982f78f6e5a0ed

Elemento del ZIP	Contenido
PDF de la evidencia	Portada, diseño, código explicado, capturas, pruebas y lista de chequeo.
proyecto/	Código fuente, configuración, esquema SQL, pantalla y pruebas.
proyecto/docs/ENDPOINTS.http	Peticiones de registro, autenticación correcta, error y validación.
ENLACE_GITHUB.txt	URL del repositorio y enlace de la ejecución verificada.
evidencias/	Capturas originales y reportes de las pruebas.
historial-github.bundle	Copia del historial Git recuperado de la ejecución.

El paquete se denomina **Laura_Tatiana_Ciro_Botero_AA5_EV01.zip**, conforme al formato indicado por la guía. Se excluyen las dependencias descargables y la base con datos de práctica. Maven y npm reconstruyen las dependencias declaradas.

El enlace de GitHub permite consultar el código y sus versiones. No es una dirección pública de la API desplegada; el servicio se ejecuta localmente en http://127.0.0.1:8080.

Paso a paso para utilizar el proyecto

1. Abrir el proyecto

Descomprimir el ZIP. Abrir la carpeta proyecto en Visual Studio Code u otro editor Java.

2. Revisar las herramientas

Tener Java 17 o superior y Maven 3.9 o superior. Node solo se necesita para repetir las pruebas del navegador.

3. Iniciar el servidor

En una terminal ubicada en proyecto ejecutar mvn spring-boot:run. Esperar a que el servidor indique que inició en el puerto 8080.

4. Abrir la pantalla

Ingresar a http://127.0.0.1:8080. No hay cuentas precargadas; primero se debe registrar una cuenta ficticia.

5. Probar el registro

Ingresar un usuario como tatiana.demo y una contraseña como Practica2026!. Pulsar Registrar usuario. Comprobar HTTP 201.

6. Probar el inicio de sesión

Escribir los mismos datos en Inicio de sesión y pulsar Comprobar credenciales. Debe aparecer Autenticación satisfactoria y HTTP 200.

7. Comprobar los errores

Cambiar la contraseña para obtener HTTP 401. Intentar registrar el mismo usuario para obtener 409. Enviar campos vacíos para obtener 400.

8. Repetir las pruebas del código

Ejecutar mvn test. Para compilar y verificar el paquete completo ejecutar mvn clean verify. README explica las pruebas adicionales del navegador.

Los datos normales quedan en data. Reiniciar el servicio conserva los usuarios. No borrar esa carpeta si contiene cuentas que se desean conservar. Para detener el programa se utiliza Ctrl+C en la terminal donde se inició.

Lista de chequeo y conclusiones

Tabla 3

Autoevaluación de GA7-220501096-AA5-EV01

Indicador	Sí	No	Evidencia de cumplimiento
1. Realiza un servicio para ser utilizado en un registro.	X		POST /api/registro; AuthController y AuthService; HTTP 201; figura 2 y pruebas.
2. Realiza un servicio para ser utilizado en un inicio de sesión.	X		POST /api/login; comparación BCrypt; HTTP 200 o 401; figuras 2 y 3.
3. Realiza las validaciones de verificación correctamente.	X		Obligatoriedad, longitudes, formato, duplicados, JSON y credenciales; tabla 1 y pruebas.
4. Utiliza herramientas de versionamiento para la creación del proyecto.	X		Git, commits, repositorio GitHub y copia del historial.
Requisitos adicionales de la guía y de la entrega.	X		Código comentado, todos los archivos del proyecto, archivo con enlace del repositorio y ZIP con nombre solicitado; PDF con portada y referencias.

Nota. Esta lista es una autoevaluación del producto y no reemplaza la valoración de la instructora. La captura enviada por la aprendiz distingue este caso AA5-EV01 de las posteriores evidencias de pruebas con Postman y del proyecto completo.

Conclusiones

El registro y la autenticación quedan separados en dos rutas reutilizables. El servidor valida la información, protege las contraseñas y devuelve respuestas identificables. Las pruebas y el historial de versiones permiten comprobar el comportamiento y continuar el trabajo en la siguiente evidencia.

Referencias

Servicio Nacional de Aprendizaje. (s. f.). *Guía de aprendizaje 7: Actividad GA7-220501096-AA5* (pp. 14-15) [Material de formación proporcionado para la actividad].

Servicio Nacional de Aprendizaje. (2026). *Estado: Arranca GA7-AA5* [Diapositiva de clase, captura proporcionada por la aprendiz].

Spring. (s. f.-a). *Building a RESTful web service*. https://spring.io/guides/gs/rest-service/

Spring. (s. f.-b). *Password storage*. Spring Security. Documentación oficial de almacenamiento de contraseñas

Elaboración del documento

Se utiliza presentación académica basada en APA: tamaño carta, márgenes de 2,54 cm, Times New Roman de 12 puntos e interlineado doble en el cuerpo, numeración de páginas, títulos de tablas y figuras y referencias con sangría francesa. Tablas, notas y código tienen tamaños ajustados para facilitar su lectura.

Los ejemplos y las capturas contienen únicamente usuarios ficticios. Los enlaces de esta evidencia permiten acceder al repositorio y a los resultados publicados; el código completo forma parte del ZIP.